

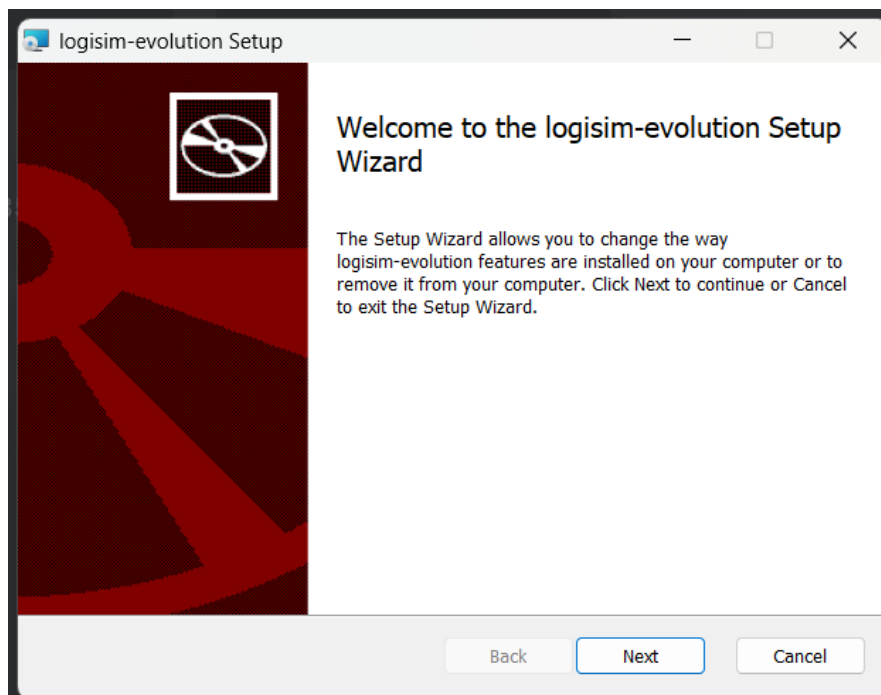
Lab 1: Introduction to Digital Logic

Software Installation Process

Prior to carrying out the digital logic experiments, the Logisim-Evolution application was set up on the computer. The steps taken to complete this installation are described below.

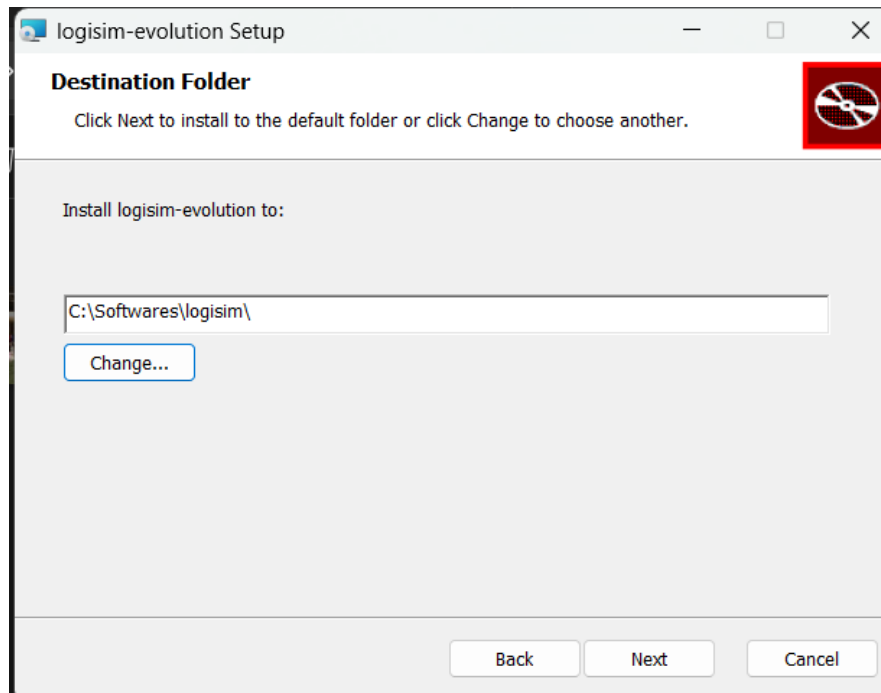
Step 1: Start the Installer

Run the Logisim-Evolution setup executable. The installation wizard opens on the welcome screen. Select **Next** to proceed.



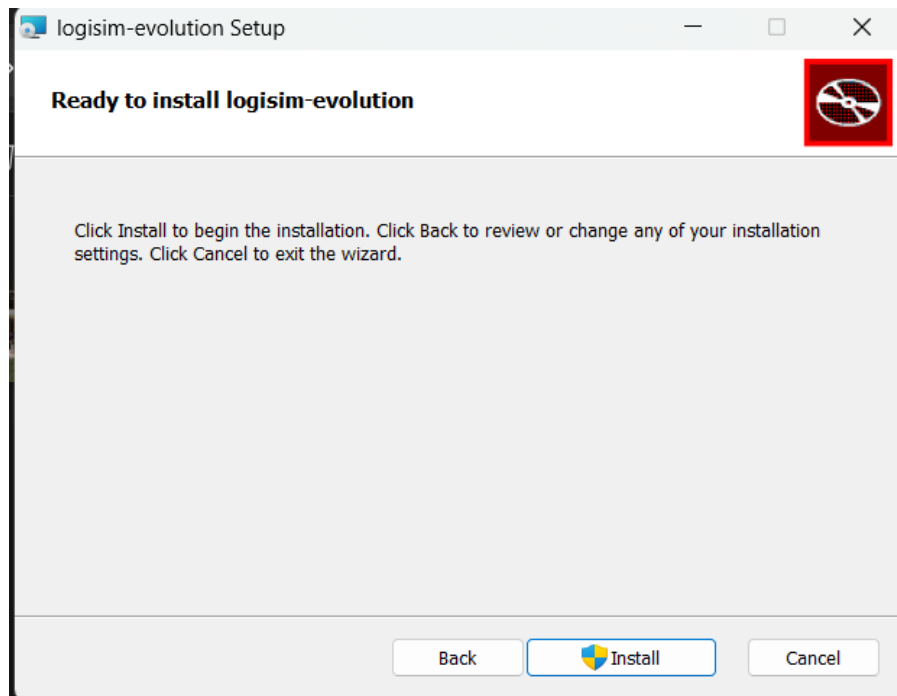
Step 2: Choose the Install Location

Pick the folder where Logisim-Evolution will be installed. The default path was left unchanged. Select **Next** to continue.



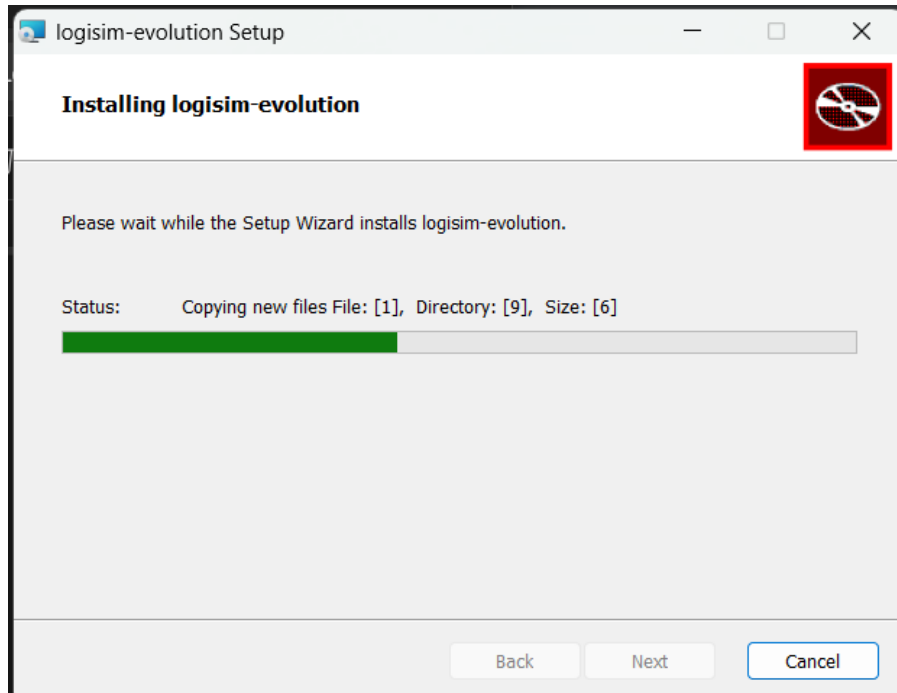
Step 3: Confirm the Installation Settings

The wizard presents a summary screen. Select the **Install** button to begin copying the program files.



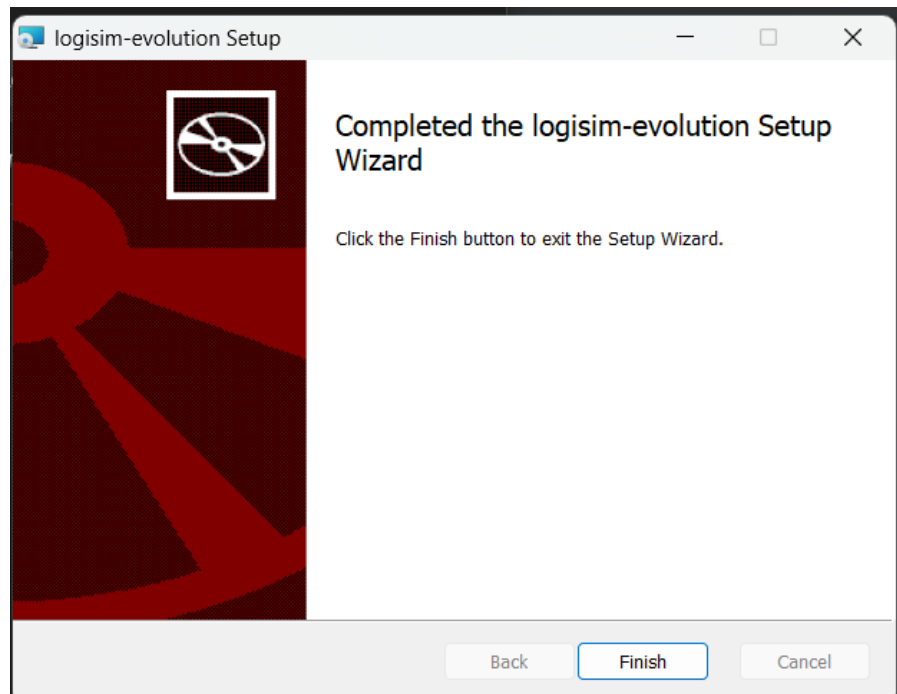
Step 4: Carry Out the Installation

The installer transfers all required files into the selected folder. Allow the process to run until the progress bar reaches 100%.



Step 5: Finish the Setup

Once the installation completes without error, select **Finish** to close the wizard.



Conclusion

Logisim-Evolution was installed without issue and was ready to be used for designing, simulating, and testing the digital logic circuits covered in this laboratory.

Experiment 1: Building an OR Gate from NOR Gates

Overview

This experiment centers on assembling an OR gate using only NOR gates. An OR gate is a core digital logic component that outputs a logical true (1) whenever at least one of its inputs is true. Because the NOR gate is a universal gate, it can be wired in different configurations to reproduce any other logic gate, including the OR gate.

Procedure

1. The NOR Gate. A NOR gate can be understood as an OR gate whose output passes through a NOT gate. It outputs a 1 only when both inputs are 0. Its behavior for two inputs is summarized in the table below.

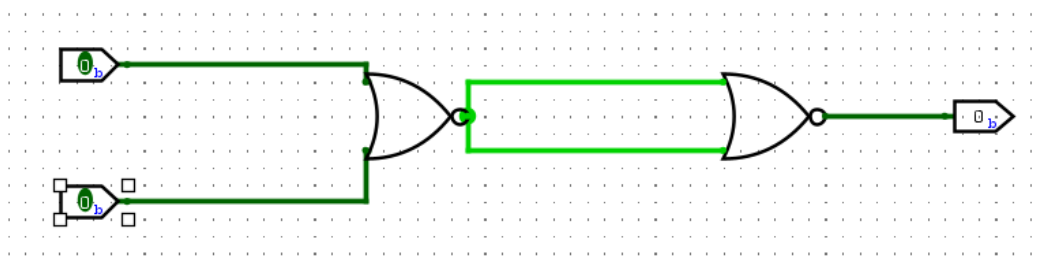
Input A	Input B	Output (A NOR B)
0	0	1
0	1	0
1	0	0
1	1	0

2. Obtaining the OR Gate. An OR gate can be derived from NOR gates using the relation:

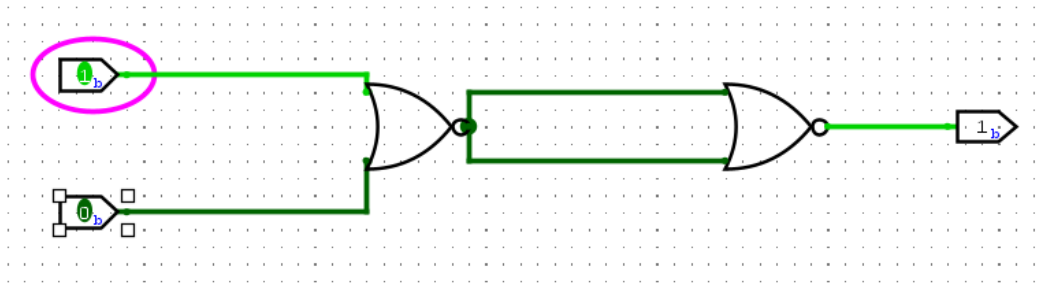
$$\text{OR}(A, B) = \text{NOT}(\text{NOR}(A, B))$$

In this arrangement, the first NOR gate computes $\text{NOR}(A, B)$, while a second NOR gate is used purely as an inverter to convert that result into the OR output.

3. Circuit Design. Connect inputs A and B to the first NOR gate.



Route the output of the first NOR gate into both inputs of a second NOR gate, which acts as an inverter.



4. Testing. Cycle through every combination of A and B and verify that the circuit's output matches that of a standard OR gate.

Conclusion

The OR gate was successfully realized using only NOR gates, confirming the universal nature of the NOR gate and its ability to substitute for other fundamental gates.

Experiment 2: Building an OR Gate from NAND Gates

Overview

The goal of this experiment is to build an OR gate using only NAND gates, another gate type classified as universal.

Procedure

1. The NAND Gate. A NAND gate behaves like an AND gate followed by an inverter, producing a 0 only when every input is 1.

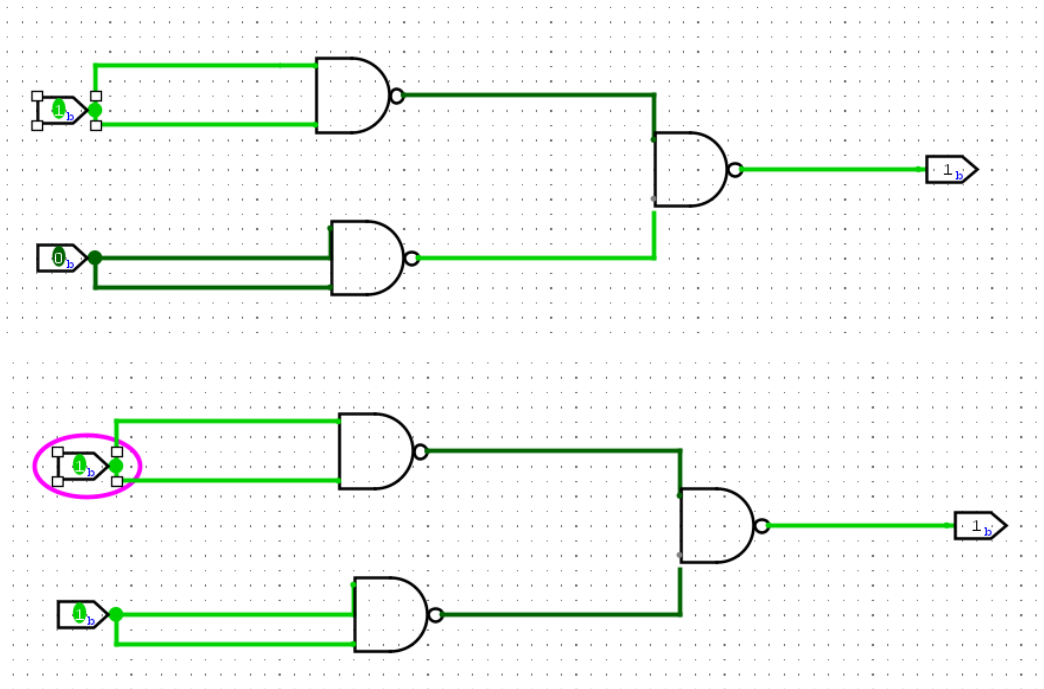
Input A	Input B	Output (A NAND B)
0	0	1
0	1	1
1	0	1
1	1	0

2. Obtaining the OR Gate. Using NAND gates, the OR function is obtained from:

$$\text{OR}(A, B) = \text{NOT}(\text{NAND}(\text{NOT}(A), \text{NOT}(B)))$$

Two NAND gates first invert A and B individually, and a third NAND gate combines the inverted signals to produce the OR output.

3. Circuit Design. Wire A and B each into a separate NAND gate configured as an inverter.



4. Testing. Apply every input combination and check the output against the expected OR truth table.

Conclusion

The circuit correctly reproduced OR-gate behavior, further illustrating how the NAND gate can be reconfigured to emulate other logic functions.

Experiment 3: Building an AND Gate from NOR Gates

Overview

This experiment implements an AND gate using only NOR gates. An AND gate outputs 1 only when both of its inputs equal 1.

Procedure

1. The AND Gate.

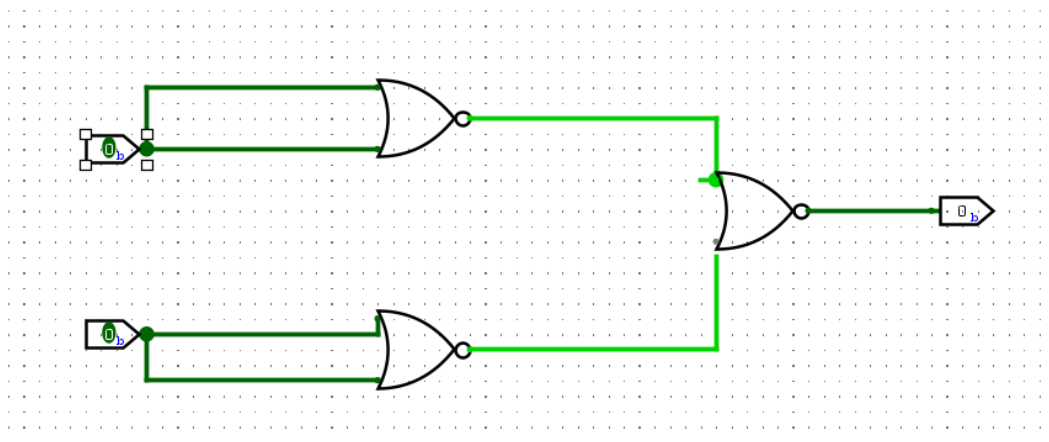
Input A	Input B	Output (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

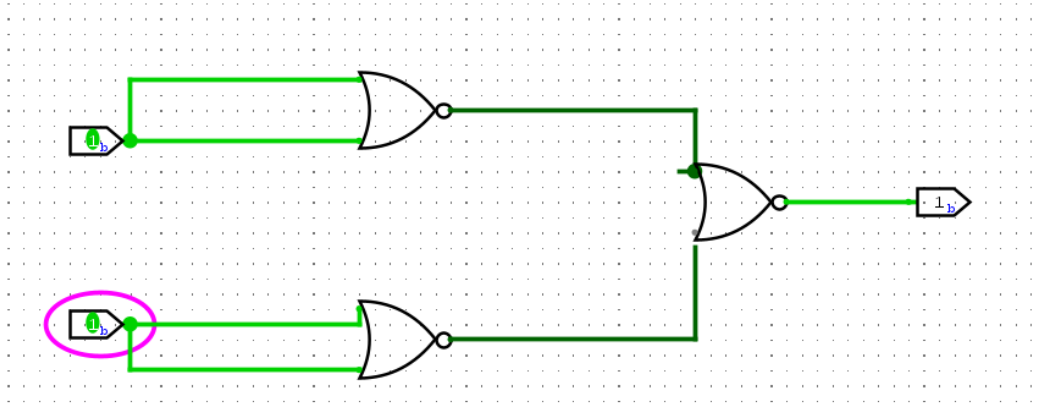
2. Obtaining the AND Gate.

$$\text{AND}(A, B) = \text{NOT}(\text{NOR}(\text{NOT}(A), \text{NOT}(B)))$$

Two NOR gates are first used to invert A and B, and a third NOR gate combines these inverted values into the final AND output.

3. Circuit Design. Invert A and B using two separate NOR gates.





4. Testing. Verify all four input combinations against the AND truth table.

Conclusion

The AND gate was successfully built from NOR gates alone, reinforcing the versatility of universal gates in digital circuit design.

Experiment 4: Building an AND Gate from NAND Gates

Overview

This experiment implements an AND gate using only NAND gates.

Procedure

1. The AND Gate.

Input A	Input B	Output (A AND B)
0	0	0
0	1	0
1	0	0
1	1	1

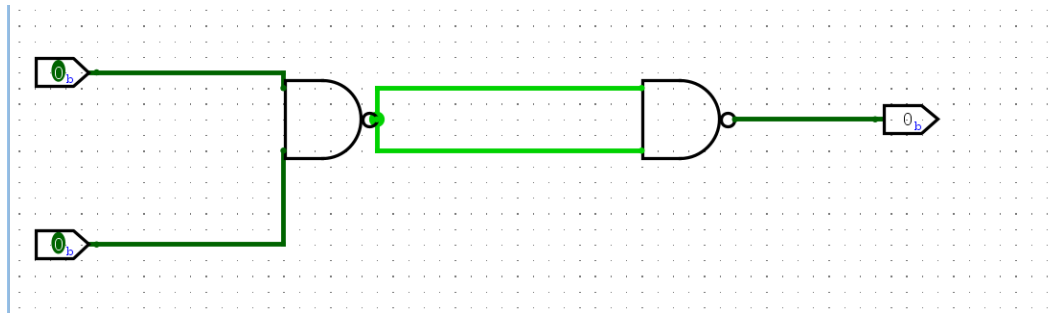
2. Obtaining the AND Gate.

$$\text{AND}(A, B) = \text{NOT}(\text{NAND}(A, B))$$

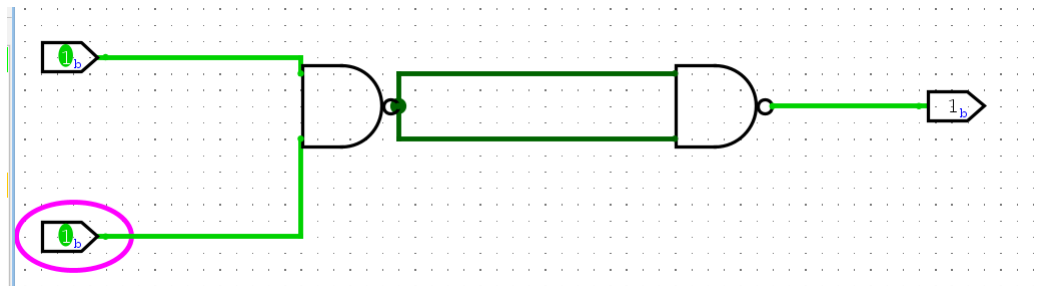
A single NAND gate computes $\text{NAND}(A, B)$, and a second NAND gate wired as an inverter converts this into the AND output.

3. Circuit Design.

Connect A and B to the first NAND gate.



Feed its output into both inputs of a second NAND gate to invert it.



4. Testing. Check every combination of A and B against the AND truth table.

Conclusion

The resulting circuit matched the expected AND behavior, again demonstrating that a single universal gate type is sufficient to build any logic function.

Experiment 5: Building a NOT Gate from NOR Gates

Overview

This experiment produces a NOT gate using only a NOR gate. A NOT gate simply inverts its single input.

Procedure

1. The NOT Gate.

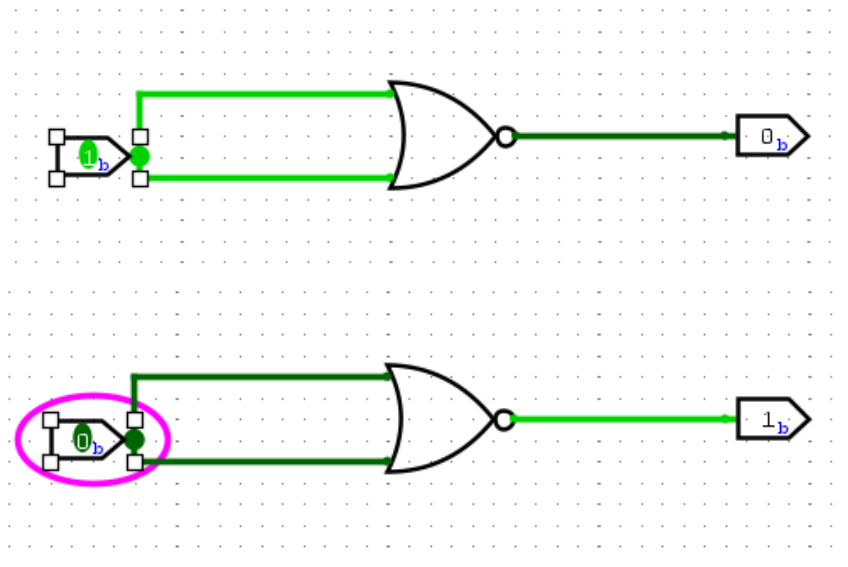
Input A	Output (NOT A)
0	1
1	0

2. Obtaining the NOT Gate.

$$\text{NOT}(A) = \text{NOR}(A, A)$$

Tying both inputs of a NOR gate to the same signal A produces an inverted version of A.

3. Circuit Design. Tie input A to both inputs of a single NOR gate.



4. Testing. Apply A = 0 and A = 1 and confirm the output is inverted in each case.

Conclusion

A functioning NOT gate was obtained from a single NOR gate, confirming the expected inverter behavior.

Experiment 6: Building a NOT Gate from NAND Gates

Overview

This experiment produces a NOT gate using only a NAND gate.

Procedure

1. The NOT Gate.

Input A	Output (NOT A)
0	1
1	0

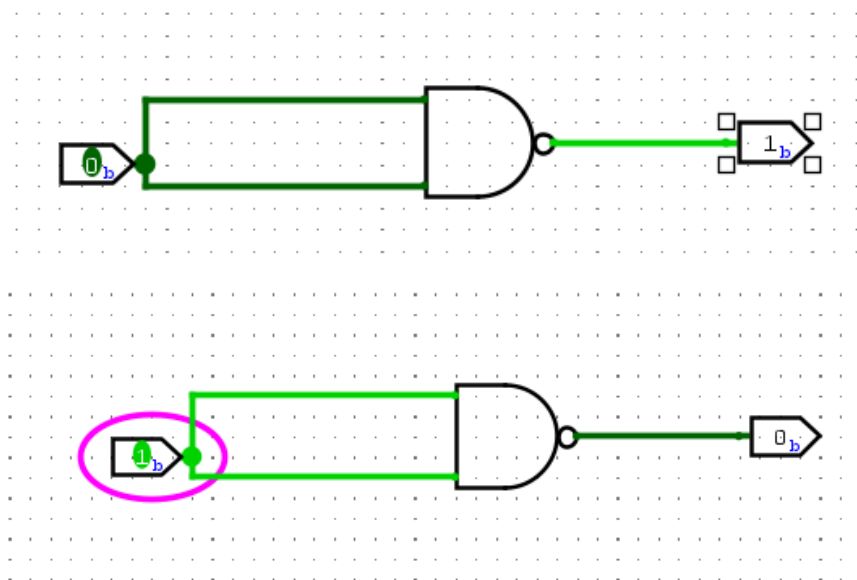
2. Obtaining the NOT Gate.

$$\text{NOT}(A) = \text{NAND}(A, A)$$

Connecting both inputs of a NAND gate to the same signal produces an inverter.

3. Circuit Design.

Tie input A to both inputs of a single NAND gate.



4. Testing.

Apply $A = 0$ and $A = 1$ and confirm correct inversion.

[Image Placeholder: Simulation output for the NOT gate]

Conclusion

The NAND-based inverter behaved exactly as a standard NOT gate would, confirming the design.

Experiment 7: Full Adder Design and Testing

Overview

This experiment builds and tests a Full Adder circuit, a component that sums three binary bits, two operand bits and a carry-in bit, to produce a sum bit and a carry-out bit.

Procedure

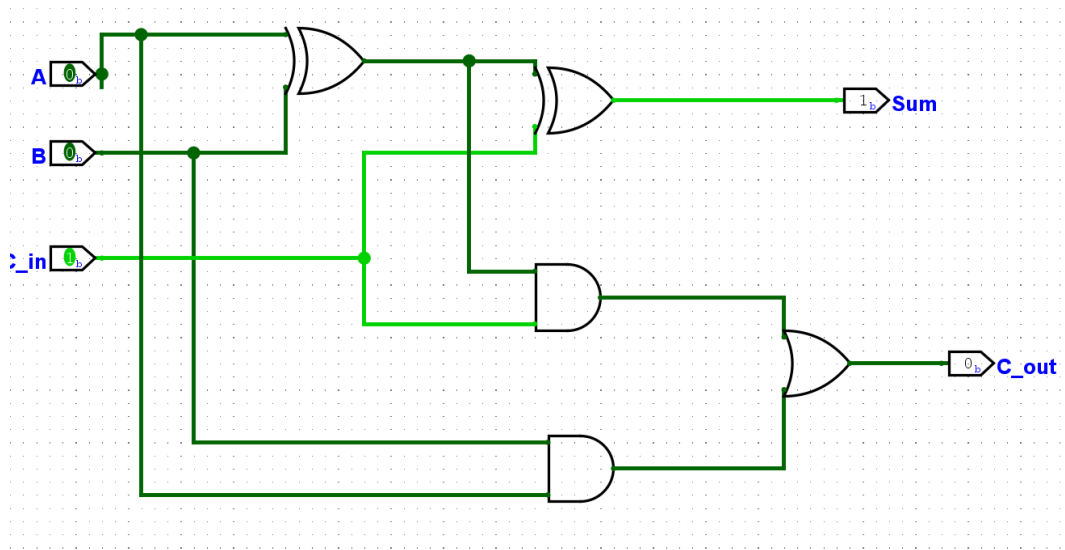
1. Full Adder Behavior. The Full Adder takes three inputs (A, B, C_in) and generates two outputs (Sum, C_out), as shown below.

Input A	Input B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2. Assembling the Full Adder. A Full Adder can be built from two Half Adders together with an OR gate:

- The first Half Adder processes A and B, producing a partial sum S1 and carry C1.
- The second Half Adder combines S1 with Cin, producing the final Sum and a second carry C2.
- The overall carry-out Cout is obtained by ORing C1 and C2.

3. Circuit Design. Feed A and B into the first Half Adder.



4. Testing. Run the circuit through all eight input combinations and confirm Sum and Cout match the truth table.

For an 8-bit test run, the overall adder reported a sum display of 10 with a carry-out of 0. Reduced to a single bit, this corresponds to Sum = 0 and Cout = 1, matching the circuit's expected behavior.

A second run reported a sum display of 11 with carry-out 0, corresponding at the single-bit level to Sum = 1 and Cout = 1, again consistent with the design.

Conclusion

Built from two Half Adders and an OR gate, the Full Adder circuit correctly carried out three-bit binary addition. This building block is central to arithmetic logic units (ALUs) and other systems that require binary addition.

Experiment 8: Binary-to-BCD Converter

Overview

This experiment implements a circuit that converts a 4-bit binary number into its Binary-Coded Decimal (BCD) equivalent, where each decimal digit is represented by its own binary code.

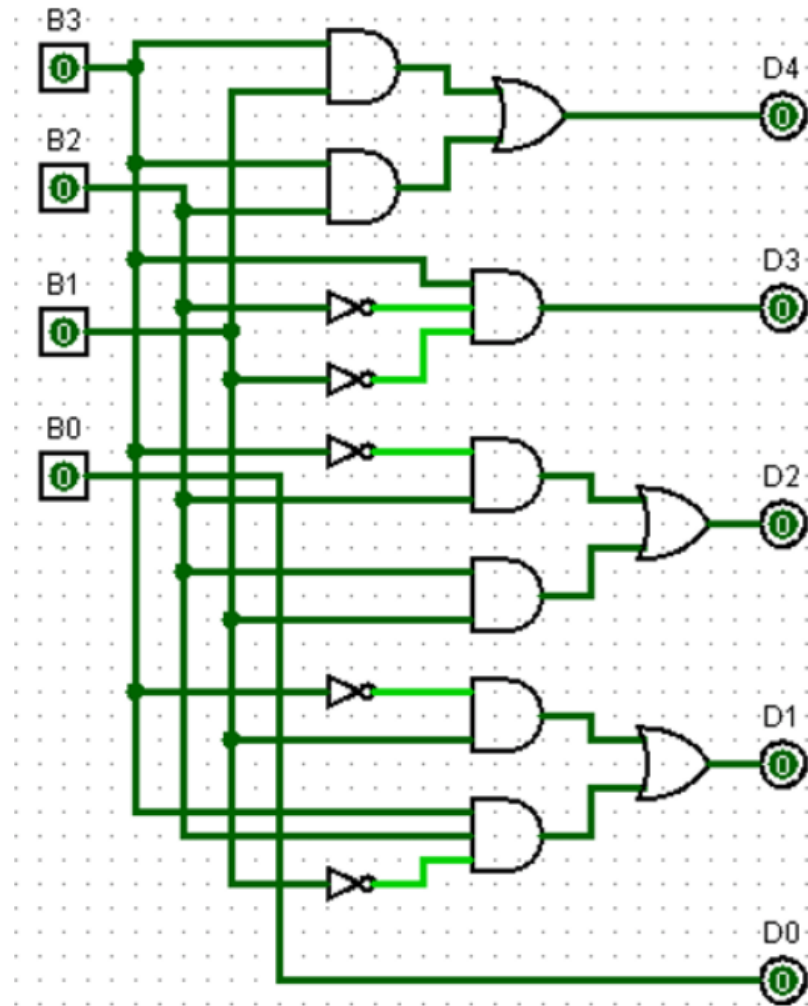
Procedure

1. Conversion Concept. A Binary-to-BCD converter accepts a binary value and outputs the corresponding BCD digits. For instance, binary 1010 (decimal 10) becomes 0001 0000 in BCD (digits “1” and “0”). The mapping for all 4-bit inputs is given below.

Binary Code				BCD Code			
B3	B2	B1	B0	D4	D3	D2	D1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1
1	0	1	0	1	0	1	0
1	0	1	1	1	0	1	1
1	1	0	0	1	1	0	0
1	1	0	1	1	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1

2. Building the Converter. The converter is realized as a network of logic gates mapping the four binary inputs to logic expressions for each BCD output bit (D4, D3, D2, D1), which can then be driven onto a seven-segment display.

3. Circuit Design. Wire the binary inputs B3, B2, B1, B0 into the converter circuit.



The converter produces the BCD outputs D4, D3, D2, D1, which feed a BCD-to-seven-segment display.

4. Testing. Sweep through binary input combinations and confirm the BCD outputs match the expected table.

Conclusion

The Binary-to-BCD converter correctly translated binary values into their BCD equivalents, demonstrating a practical application of digital number-system conversion in devices such as calculators and digital displays.

End of Lab 1 Report